Preguntas y respuestas para la defensa individual

CAT-IA · ISY0101 · Ingeniería de Soluciones con IA · 21/10/2026. Ambas integrantes deben poder explicar todo el proyecto. Las respuestas describen la implementación, no una autonomía ilimitada.

1. ¿Qué problema organizacional resuelven?

Ayudamos al operador de la empresa ficticia Nexo TI a consultar procedimientos, continuar un caso y preparar una respuesta con evidencia. No medimos una reducción real de tiempos de una empresa; el beneficio es propuesto y el comportamiento se prueba con casos simulados.

2. ¿Qué es un LLM y cuál usan?

Un modelo de lenguaje genera texto a partir del contexto. Usamos qwen2.5:1.5b mediante Ollama local. No lo entrenamos ni modificamos sus pesos. La configuración del modelo está en .env y la llamada en catia/generation.py.

3. ¿Qué diferencia hay entre prompt, memoria y entrenamiento?

El prompt da instrucciones para una ejecución. La memoria aporta antecedentes recuperables. Entrenar modifica parámetros del modelo. Cambiar el prompt o recordar un ticket no significa entrenar nuestra IA.

4. ¿Qué es RAG?

Recuperar fragmentos relevantes y entregarlos al modelo para generar una respuesta respaldada. En CAT-IA el flujo pasa por Retriever.search, la herramienta de búsqueda y el generador. Si faltan fuentes, no debemos completar la respuesta con hechos inventados.

5. ¿Cuáles son sus fuentes internas y externas?

Siete documentos internos simulados de Nexo TI, como facturación y conectividad; dos síntesis de guías de Mozilla. data/sources.json registra origen, versión y SHA-256. No hay conexión a cuentas de Microsoft o Gmail ni navegación web en cada consulta.

6. ¿Por qué falló «me cobraron dos veces»?

TF-IDF compara términos y sus pesos; esa paráfrasis no compartía suficiente vocabulario con el documento. Se agregó expansión explícita y se compara con recuperación híbrida usando embeddings. Son soluciones distintas y el informe registra sus resultados.

7. ¿Qué es un embedding?

Un vector numérico producido por un modelo para representar texto. Comparamos vectores normalizados mediante producto escalar, equivalente al coseno. La similitud orienta la búsqueda, pero no es probabilidad de verdad. Usamos nomic-embed-text; su utilidad en español debe medirse con nuestros casos.

8. ¿Cómo funcionan las memorias?

La corta guarda tres intercambios y un resumen extractivo. La persistente conserva hasta veinte registros por hilo en case_notes. En modo híbrido se recuperan hasta dos entradas humanas con similitud de al menos 0,65. Se usa el mismo UUID y se conserva al reiniciar; no se buscan datos de otro hilo. SQLite guarda datos; no calcula por sí mismo los embeddings.

9. ¿Por qué no usan respuestas anteriores como evidencia?

Un borrador anterior puede contener errores. Guardarlo no lo convierte en un hecho. La búsqueda de memoria devuelve entradas humanas como antecedentes no confiables; solo los documentos recuperados pueden citarse.

10. ¿Qué cambió ante la consulta de VPN después de factura?

select_context detecta señales temáticas conocidas, ignora antecedentes incompatibles y lo muestra en pantalla. Las continuaciones posteriores se restringen al segmento reciente. Las consultas mixtas o sin palabras reconocidas pueden escapar al control; borrar el hilo sigue siendo útil.

11. ¿El modelo pequeño causó el error?

No lo demostramos. Para afirmarlo necesitaríamos comparar modelos bajo los mismos documentos, consultas, configuración y criterios de calidad. La retroalimentación evidencia contaminación de contexto y una validación insuficiente; no identifica por sí sola una causa asociada al tamaño.

12. ¿Dónde está la herramienta de escritura?

tool_draft_response en catia/tools.py genera, valida y registra el borrador mediante MemoryStore.save_draft. Escribe en case_notes y devuelve saved_draft_id. No envía un correo ni modifica sistemas externos. Si la validación falla, no registra un borrador aprobado.

13. ¿Qué decide el modelo y qué decide el código?

En ReAct el modelo elige el orden inicial de búsqueda o SLA y redacta. El código fija herramientas permitidas, argumentos, precondiciones, tres llamadas, política, contexto y validación. CrewAI organiza roles en secuencia, cada uno con una herramienta permitida. La persona aprueba la respuesta final.

14. ¿Qué es ReAct aquí?

Un ciclo acotado de observar, seleccionar una acción, ejecutar una herramienta y usar su resultado. No exponemos pensamientos internos; mostramos acciones y observaciones. Es una implementación limitada sobre herramientas LangChain, no un planificador abierto.

15. ¿Por qué LangChain y CrewAI?

LangChain aporta contratos de herramientas con validación. CrewAI ofrece Agent, Task y Process.sequential para comparar un equipo de roles. No necesitamos dos frameworks para todo problema; aquí justificamos la comparación académica. CrewAI añade dependencias y llamadas; medir latencia permite discutir su costo.

16. ¿Cómo evitan bucles y acciones peligrosas?

Existe una lista permitida, no se repite una herramienta y el borrador requiere búsqueda y política previas. ReAct limita tres herramientas; CrewAI limita iteraciones y llamadas. No hay herramienta de ejecución arbitraria ni envío a clientes.

17. ¿Una cita real garantiza fidelidad?

No. validate_evidence comprueba que exista el fragmento y que la cita sea literal. validate_alignment añade consistencia temática. Ninguna prueba todas las afirmaciones ni detecta necesariamente una promesa falsa de reembolso. La fidelidad requiere revisar si lo dicho se desprende realmente de la fuente.

18. ¿Qué significan precisión y recall de recuperación?

Precisión: proporción de documentos recuperados que son relevantes. Recall: proporción de documentos relevantes esperados que fueron recuperados. Requieren referencias etiquetadas. No son lo mismo que exactitud de categoría ni que fidelidad del texto generado.

19. ¿Qué ocurre si falla una herramienta?

Se registra un evento de error y se devuelve una salida controlada. Sin contexto se abstiene. Ante desalineación devuelve evidence_mismatch. El respaldo a demo solo se activa si el operador lo permite y se etiqueta como simulación; no se cuenta como éxito de Ollama.

20. ¿Qué pruebas muestran decisiones adaptativas?

Continuidad de facturación, cambio a VPN con descarte de contexto, filtro que excluye INT-FAC y falla de proveedor inyectada. tests/test_feedback.py comprueba esos comportamientos. reports/feedback_real.json registra inferencias reales separadas de los tests simulados.

21. ¿Por qué SQLite y no una base vectorial externa?

El volumen es pequeño y SQLite simplifica la instalación. Los embeddings se calculan con Ollama y se comparan en memoria; los registros persisten en SQLite. Un volumen grande justificaría índices vectoriales, control de acceso, migraciones y retención más robusta. No afirmamos escalabilidad empresarial actual.

22. ¿Qué muestra la traza?

Herramienta, actor, resultado resumido, estado y duración, además de identificadores. No es razonamiento privado ni certeza. El JSON descargado permite relacionar el resultado con fuentes, controles y proveedor.

23. ¿Qué implica un resultado draft?

Que pasó los controles implementados y está pendiente de revisión. No equivale a respuesta verdadera, enviada o aprobada. Un error factual dentro del mismo tema puede pasar los controles heurísticos.

24. ¿Cómo responderías una tarea nueva del docente?

Primero aclaro el requerimiento y el modo. Ejecuto, inspecciono fuentes y herramientas, comparo respuesta con evidencia y explico la rama seguida. Si falla, identifico la etapa por status y eventos y propongo una corrección concreta. No invento una causa basándome solo en el texto final.